Week 3 – Process 2

Operating Systems Ch 3 (Sections 3.4 – 3.8)

Korea University Sejong Campus, Department of Computer Science & Software

Table of Contents

0. **Quiz** – Beginning of 1st Hour
1. Interprocess Communication (IPC) Overview
2. Shared-Memory Systems
3. Message-Passing Systems
4. IPC Examples (POSIX, Mach, Windows)
5. Pipes (Ordinary / Named)
6. Client-Server Communication (Sockets, RPC)
7. Lab – pipe, POSIX shared memory
8. Summary and Next Week Preview

Part 1

Interprocess Communication (IPC)

Silberschatz Ch 3.4

Independent vs Cooperating Processes

Processes executing concurrently in an operating system fall into two categories:

Independent process

- Cannot affect or be affected by the execution of other processes
- Does not share data with other processes

Cooperating process

- Can affect or be affected by the execution of other processes
- Any process that shares data with others falls into this category

> Most real-world systems consist of cooperating processes.

Reasons for Process Cooperation

Four reasons why cooperating processes are needed:

1. **Information sharing**
   - Multiple processes accessing the same data concurrently (e.g., copy-paste)
2. **Computation speedup**
   - Improve processing speed by executing tasks in parallel
   - Effective on multicore systems
3. **Modularity**
   - Design system functions as separate processes/threads
4. **Convenience**
   - Users perform multiple tasks simultaneously (editing, playing music, web browsing, etc.)

What is IPC?

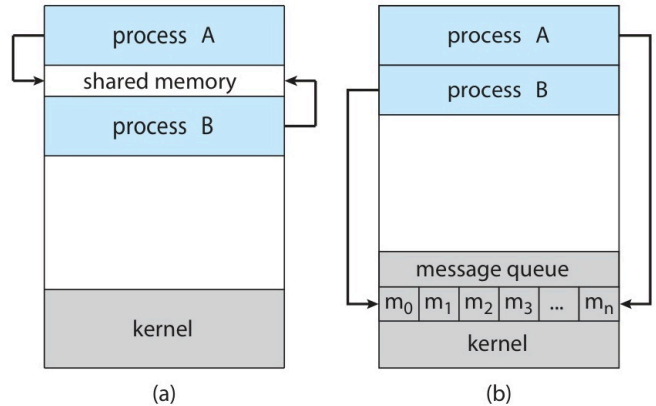
Interprocess Communication (IPC)

- A mechanism for cooperating processes to exchange data
- "send data to and receive data from each other"

Two fundamental IPC models:

	Shared Memory	Message Passing
Method	Read/write to a shared memory region	Send and receive messages
Performance	Fast (minimal kernel involvement)	Relatively slow (system calls)
Synchronization	Programmer must handle it directly	Managed by the OS
Suitable for	Large data exchanges	Small data, distributed systems

Two IPC Models (Figure 3.11)



Silberschatz, Figure 3.11 – (a) Shared memory. (b) Message passing.

- Shared memory: The kernel only sets up the shared region; subsequent access is normal memory access
- Message passing: The kernel (system calls) is involved in every communication

Chrome Browser: Multiprocess Architecture

Chrome is a representative multiprocess architecture utilizing IPC:

- **Browser process** (1) – Manages UI, disk/network I/O
- **Renderer process** (per tab) – Renders web pages (HTML, JS, images)
- **Plug-in process** (per plug-in) – Flash, QuickTime, etc.

Advantages:

- If one website crashes, only that renderer is affected → other tabs continue to work normally
- Renderers run in a **sandbox** → enhanced security
- Processes communicate via IPC

> In single-process browsers, one tab crash would bring down the entire browser.

Part 2

Shared-Memory Systems

Silberschatz Ch 3.5

Shared-Memory IPC

How shared-memory IPC works:

1. Processes that want to communicate **establish a shared memory region**
2. The shared region typically resides in the address space of the creating process
3. Other processes **attach** the shared memory to their own address spaces
4. Data is exchanged through read/write operations

Key characteristics:

- The OS normally **prohibits** inter-process memory access
- To use shared memory, this restriction must be **explicitly removed**
- The format and location of data are **determined by the processes** (the OS is not involved)
- **Synchronization is the programmer's responsibility** (e.g., preventing concurrent writes)

Producer-Consumer Problem

A classic paradigm for cooperating processes:

- **Producer** – A process that produces information
- **Consumer** – A process that consumes information

Real-world examples:

- Compiler (producer) → Assembler (consumer)
- Web server (producer) → Web browser (consumer)

Solution: Use a **shared buffer**

- The producer fills the buffer
- The consumer empties the buffer
- The producer and consumer must be **synchronized**

Two Types of Buffers

Unbounded buffer

- No practical limit on buffer size
- Consumer: may wait for new items to arrive
- Producer: can always produce new items

Bounded buffer

- Buffer size is **fixed**
- Consumer: waits if the buffer is empty
- Producer: waits if the buffer is full

> In practice, bounded buffers are far more common, and they give rise to synchronization issues.

Bounded Buffer: Shared Data Structure

Data shared between Producer and Consumer:

```
#define BUFFER_SIZE 10

typedef struct {
    /* ... */
} item;

item buffer[BUFFER_SIZE];
int in = 0;     /* next write position (producer) */
int out = 0;    /* next read position (consumer) */
```

- Implemented as a **circular array** – uses two pointers: in and out
- Buffer empty: `in == out`
- Buffer full: `((in + 1) % BUFFER_SIZE) == out`
- With this scheme, at most **BUFFER_SIZE - 1** items can be stored

Bounded Buffer: Producer (Figure 3.12)

```
item next_produced;

while (true) {
    /* produce an item in next_produced */

    while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing -- buffer is full, busy waiting */

    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
}
```

How it works:

1. Produce an item
2. **Busy wait** until space becomes available in the buffer
3. Store the item in `buffer[in]`
4. Advance the `in` pointer to the next position (circular)

Bounded Buffer: Consumer (Figure 3.13)

```
item next_consumed;

while (true) {
    while (in == out)
        ; /* do nothing -- buffer is empty, busy waiting */

    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;

    /* consume the item in next_consumed */
}
```

How it works:

1. **Busy wait** until an item appears in the buffer
2. Retrieve the item from `buffer[out]`
3. Advance the `out` pointer to the next position (circular)
4. Consume the item

> This example does not address concurrent access issues → covered in Ch 6, 7

Part 3

Message-Passing Systems

Silberschatz Ch 3.6

Message Passing Overview

A method for inter-process communication without shared memory

Two basic operations:

- **send(message)** – send a message
- **receive(message)** – receive a message

Characteristics:

- Does not share the same address space
- Particularly useful in **distributed environments** (different computers connected via a network)
- Example: internet chat programs

Three considerations for **logical implementation** of communication links:

1. **Naming** – How do you identify the communication partner?
2. **Synchronization** – Synchronous or asynchronous?
3. **Buffering** – What is the capacity of the message queue?

Naming: Direct Communication

Direct Communication – Explicitly specify the communication partner

Symmetric:

- `send(P, message)` – send a message to process P
- `receive(Q, message)` – receive a message from process Q
- Both sides specify their partner

Asymmetric:

- `send(P, message)` – send to process P
- `receive(id, message)` – receive from any process; id is set to the sender

Properties of direct communication:

- **Exactly one link** exists between each pair of processes
- Links are established **automatically**
- Drawback: if a process identifier changes, all references must be updated → **limits modularity**

Naming: Indirect Communication

Indirect Communication – Exchange messages through a **mailbox (port)**

- `send(A, message)` – send a message to mailbox A
- `receive(A, message)` – receive a message from mailbox A

Properties of indirect communication:

- Two processes must share a **common mailbox** for a link to exist
- A single link can be associated with **two or more** processes
- **Multiple links** can exist between a pair of processes (each through a different mailbox)

Mailbox ownership:

- **Process-owned**: only the owner can receive; mailbox is destroyed when the process terminates
- **OS-owned**: exists independently; the OS provides system calls for creation, deletion, send, and receive

Naming: Mailbox Multiple Receiver Problem

If P1 sends a message to mailbox A, and both P2 and P3 call receive() on A, who gets it?

```
P1 ──send(A, msg)──▷ [ Mailbox A ] ◁──receive(A, msg)── P2
                                   ◁──receive(A, msg)── P3
```

Possible solutions:

1. Allow a link to be associated with **at most two processes**
2. Allow only **one process at a time** to execute receive()
3. **The system arbitrarily selects** a receiver (e.g., round robin) and notifies the sender of who received the message

Synchronization

send() and receive() can be either **blocking (synchronous)** or **non-blocking (asynchronous)**:

Operation	Blocking (Synchronous)	Non-blocking (Asynchronous)
send	Wait until the receiver accepts	Send and proceed immediately
receive	Wait until a message arrives	Receive if available, otherwise return null

Rendezvous

- Combination of blocking send + blocking receive
- The sender and receiver wait for each other
- The producer-consumer problem becomes trivially solvable

Rendezvous: Producer-Consumer (Figures 3.14, 3.15)

Implementation using blocking send + blocking receive:

```
/* Producer */
message next_produced;

while (true) {
    /* produce an item in next_produced */
    send(next_produced);    /* blocking: waits until received */
}
```

```
/* Consumer */
message next_consumed;

while (true) {
    receive(next_consumed); /* blocking: waits until a message arrives */
    /* consume the item in next_consumed */
}
```

No shared buffer is needed – the OS manages message delivery

Buffering

Messages are stored in a **temporary queue** attached to the communication link.

Three approaches based on queue capacity:

1. Zero capacity (capacity 0)

- No pending messages in the queue
- The sender **must wait** until the receiver accepts (rendezvous)

2. Bounded capacity (finite capacity n)

- Can store at most n messages
- If the queue is not full, the sender proceeds immediately
- If the queue is full, the sender **waits**

3. Unbounded capacity (infinite capacity)

- Messages can queue indefinitely
- The sender **never waits**

Part 4

IPC Examples

Silberschatz Ch 3.7

POSIX Shared Memory

POSIX provides a shared memory API based on **memory-mapped files**.

Three-step procedure:

Step 1: Create a shared memory object

```
int fd = shm_open(name, O_CREAT | O_RDWR, 0666);
```

- `name`: name of the shared memory object (processes access it using the same name) - `O_CREAT`: create if it does not exist - `O_RDWR`: allow both reading and writing - Return value: file descriptor (integer)

Step 2: Set the size

```
ftruncate(fd, 4096);  /* 4096 bytes */
```

Step 3: Map to the address space

```
char *ptr = (char *)mmap(0, 4096, PROT_READ | PROT_WRITE,
                         MAP_SHARED, fd, 0);
```

POSIX Shared Memory – Producer (Figure 3.16)

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>

int main()
{
    const int SIZE = 4096;
    const char *name = "OS";
    const char *message_0 = "Hello";
    const char *message_1 = "World!";

    int fd;
    char *ptr;

    fd = shm_open(name, O_CREAT | O_RDWR, 0666);
    ftruncate(fd, SIZE);
    ptr = (char *)mmap(0, SIZE, PROT_READ | PROT_WRITE,
                       MAP_SHARED, fd, 0);

    sprintf(ptr, "%s", message_0);
```

POSIX Shared Memory – Consumer (Figure 3.17)

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>

int main()
{
    const int SIZE = 4096;
    const char *name = "OS";

    int fd;
    char *ptr;

    /* Open shared memory (read-only) */
    fd = shm_open(name, O_RDONLY, 0666);

    /* Map to memory */
    ptr = (char *)mmap(0, SIZE, PROT_READ, MAP_SHARED, fd, 0);

    /* Read */
    printf("%s", (char *)ptr);
```

POSIX Shared Memory – Flow Summary

```
Producer                                Consumer
────────                                ────────
shm_open("OS", O_CREAT|O_RDWR)
ftruncate(fd, 4096)
mmap( ... , MAP_SHARED, fd, 0)
     ↓
sprintf(ptr, "Hello")
sprintf(ptr, "World!")
     ↓
Exit                                    shm_open("OS", O_RDONLY)
                                        mmap( ... , MAP_SHARED, fd, 0)
                                             ↓
                                        printf("%s", ptr)
                                        → Outputs "HelloWorld!"
                                             ↓
                                        shm_unlink("OS")
```

- The producer runs first and writes the data
- The consumer opens the shared memory with the same name and reads the data
- The consumer cleans up with `shm_unlink()`

Mach Message Passing

Mach – The microkernel underlying macOS and iOS

Core concept: **Everything is a message**

- Even system calls are implemented as messages
- All communication between processes (tasks) and threads uses message passing

Port (mailbox):

- A message queue with finite capacity
- **Unidirectional** – bidirectional communication requires a separate reply port
- Multiple senders allowed, but **only one receiver**

Special Ports:

- **Task Self port** – used to send messages to the kernel
- **Notify port** – used by the kernel to notify the task of events

Mach: Port Creation and Message Passing

Port creation:

```
mach_port_t port;
mach_port_allocate(
    mach_task_self(),           /* reference to the task itself */
    MACH_PORT_RIGHT_RECEIVE,    /* receive right */
    &port);                     /* port name */
```

Message structure:

- Fixed-size header: message size, source/destination port
- Variable-size body: actual data

Sending/receiving messages:

```
/* Send */
mach_msg(&message.header, MACH_SEND_MSG,
         sizeof(message), 0, MACH_PORT_NULL,
         MACH_MSG_TIMEOUT_NONE, MACH_PORT_NULL);

/* Receive */
mach_msg(&message.header, MACH_RCV_MSG,
```

Mach: When the Queue is Full

When a port's message queue is full, the sender has the following options:

1. **Wait indefinitely** – until space becomes available in the queue
2. **Wait at most n milliseconds** – with a timeout
3. **Return immediately** – do not wait
4. **Temporarily cache** – entrust the message to the OS
   - Receive a notification when space becomes available
   - Only one pending message per thread is allowed

Performance optimization:

- Messages within the same system: use **virtual memory mapping**
- Map the sender's address space to the receiver → no actual copy → significant performance improvement

Windows – ALPC (Advanced Local Procedure Call)

A mechanism for inter-process communication within the same machine on Windows

Port types:

- **Connection port** – published by the server, visible to all processes
- **Communication port** – a dedicated channel after connection

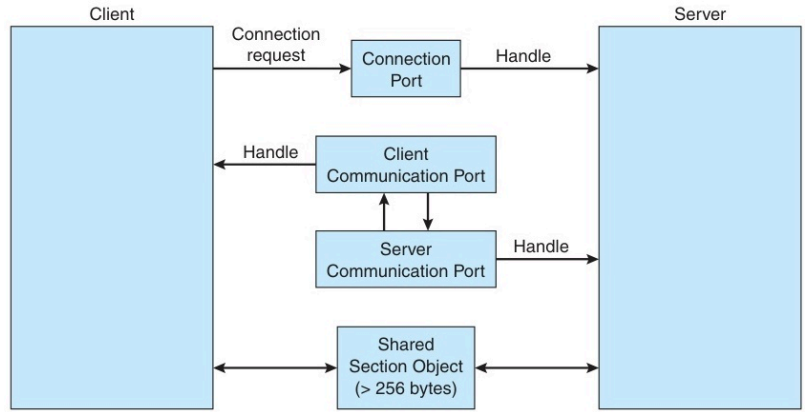
Connection procedure:

1. The client requests a connection to the server's connection port
2. The server creates a **channel** → communication ports for both client and server
3. Bidirectional communication through the channel

Message delivery methods (based on size):

1. **Small messages (<=256 bytes)** – use the port's message queue, delivered by copy
2. **Large messages** – use a **section object** (shared memory)
3. **Very large data** – the server reads/writes directly in the client's address space

Windows ALPC Structure (Figure 3.19)



Silberschatz, Figure 3.19 – Advanced local procedure calls in Windows

- ALPC is not directly exposed through the Windows API
- Applications use standard RPC, and ALPC handles it internally when on the same system

Part 5

Pipes

Silberschatz Ch 3.7.4

Pipes Overview

Pipe = A **conduit** through which two processes can communicate

- One of the oldest IPC mechanisms, present since early UNIX systems

Considerations when implementing pipes:

1. Bidirectional or unidirectional?
2. If bidirectional, half-duplex or full-duplex?
3. Is a parent-child relationship required?
4. Can it communicate over a network?

Two types:

- **Ordinary pipe** – requires parent-child relationship, unidirectional, destroyed when the process terminates
- **Named pipe (FIFO)** – unrelated processes can communicate, bidirectional possible, exists in the file system

Ordinary Pipes

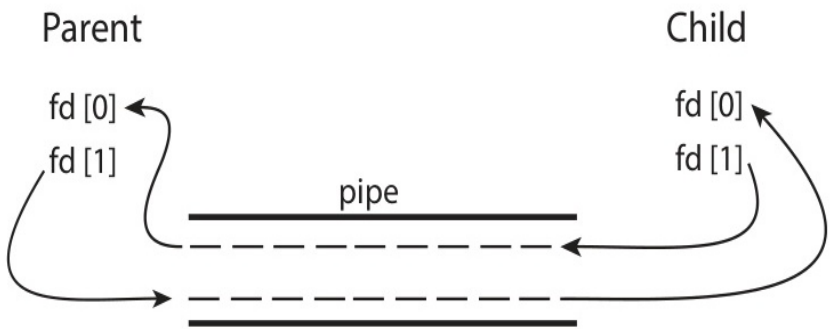
Standard producer-consumer pattern:

- Producer writes to the **write end**
- Consumer reads from the **read end**
- **Unidirectional** – use two pipes for bidirectional

Creation in UNIX:

```
int fd[2];
pipe(fd);
```

- `fd[0]` – **read end** (read-only) - `fd[1]` – **write end** (write-only) - A pipe is a special file → accessed via `read()`, `write()` - The child **inherits** the pipe through fork()



Silberschatz, Figure 3.23 – File descriptors for ordinary pipes

Ordinary Pipe – Code Example (Figures 3.21-3.22)

```c
#include <sys/types.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

#define BUFFER_SIZE 25
#define READ_END    0
#define WRITE_END   1

int main(void)
{
    char write_msg[BUFFER_SIZE] = "Greetings";
    char read_msg[BUFFER_SIZE];
    int fd[2];
    pid_t pid;

    if (pipe(fd) == -1) {
        fprintf(stderr, "Pipe failed");
        return 1;
    }

    pid = fork();

    if (pid > 0) {          /* Parent process */
```

Ordinary Pipe – Key Points

Rules that must be followed:

1. **Unused ends must be closed with close()**
   - If the parent only writes → `close(fd[READ_END])`
   - If the child only reads → `close(fd[WRITE_END])`
2. **Reason**: Essential for **EOF detection** in the pipe
   - The reader receives 0 (EOF) from `read()` only when all writers have closed the write end
   - If not closed, the reader may be stuck in a **perpetual blocking** state
3. **Process scope**:
   - An ordinary pipe **cannot be accessed from outside** the creating process
   - It can only be shared by inheriting it through fork()

Pipes in Practice – Shell Usage

Pipes using the `|` character in UNIX shells:

```
ls -l | less
```

- `ls -l` (producer) – outputs the directory listing
- `less` (consumer) – displays one screen at a time
- stdout of ls is passed to stdin of less through the pipe

```
cat file.txt | grep "error" | wc -l
```

- A **pipe chain** that finds lines containing "error" in file.txt and counts them
- Each command runs as a separate **process**

Similar on Windows:

```
dir | more
```

Named Pipes (FIFO)

Overcoming the limitations of ordinary pipes:

Property	Ordinary Pipe	Named Pipe (FIFO)
Process relationship	Parent-child required	No relationship needed
Lifetime	Destroyed when process terminates	Exists in the file system, persists until explicitly deleted
Direction	Unidirectional	Bidirectional possible (UNIX: half-duplex)
Identification	fd[0], fd[1]	File system path

Usage in UNIX:

```
/* Creation */
mkfifo("/tmp/my_fifo", 0666);
```

Named Pipes – UNIX vs Windows

UNIX Named Pipes (FIFOs):

- Created with the `mkfifo()` system call
- Manipulated with `open()`, `read()`, `write()`, `close()`
- **Half-duplex** – two FIFOs needed for bidirectional data transfer
- Transmits only **byte-oriented** data
- Communication limited to the **same machine**

Windows Named Pipes:

- `CreateNamedPipe()`, `ConnectNamedPipe()`
- `ReadFile()`, `WriteFile()`
- Supports **full-duplex**
- Supports both **byte-oriented and message-oriented** data
- Can communicate **between different machines**

Part 6

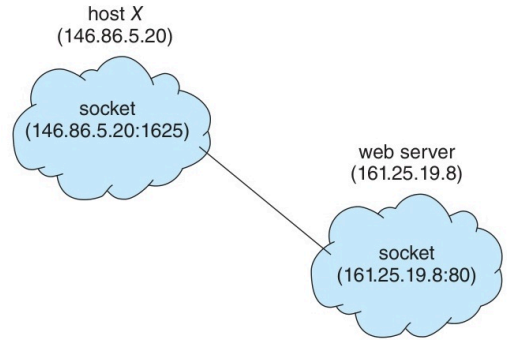
Client-Server Communication

Silberschatz Ch 3.8

Sockets

Socket = An **endpoint** for communication

- Communication occurs through a pair of sockets (one per process)
- Identified by **IP address + Port number**



Silberschatz, Figure 3.26 – Communication using sockets

- Client: assigned an arbitrary port greater than 1024 (e.g., 1625)
- Server: waits on a **well-known port** (HTTP=80, SSH=22, FTP=21)
- Every connection consists of a **unique pair of sockets**

Socket: Well-Known Ports and Loopback

Well-known ports (below 1024):

Port	Service
22	SSH
21	FTP
80	HTTP
443	HTTPS

- Ports 1024 and above: assigned arbitrarily to clients

Loopback address: 127.0.0.1

- A special IP address that refers to the local machine itself

Socket Example – Date Server (Java, Figure 3.27)

```java
import java.net.*;
import java.io.*;

public class DateServer {
    public static void main(String[] args) {
        try {
            ServerSocket sock = new ServerSocket(6013);

            /* Wait for client connections */
            while (true) {
                Socket client = sock.accept();
                PrintWriter pout = new
                    PrintWriter(client.getOutputStream(), true);

                /* Send current date to socket */
                pout.println(new java.util.Date().toString());

                /* Close socket and wait again */
                client.close();
            }
        }
        catch (IOException ioe) {
            System.err.println(ioe);
        }
```

Socket Example – Date Client (Java, Figure 3.28)

```java
import java.net.*;
import java.io.*;

public class DateClient {
    public static void main(String[] args) {
        try {
            /* Connect to server (loopback, port 6013) */
            Socket sock = new Socket("127.0.0.1", 6013);

            InputStream in = sock.getInputStream();
            BufferedReader bin = new
                BufferedReader(new InputStreamReader(in));

            /* Read date from socket */
            String line;
            while ((line = bin.readLine()) ≠ null)
                System.out.println(line);

            /* Close socket */
            sock.close();
        }
        catch (IOException ioe) {
            System.err.println(ioe);
        }
```

Limitations of Sockets

Socket-based communication is efficient but **low-level**:

- Only exchanges an **unstructured stream of bytes**
- Imposing structure on data is the **application's responsibility**
- Serialization, protocol design, etc. are required

> A higher level of abstraction is needed → **Remote Procedure Calls (RPC)**

Key idea of RPC:

- Use remote function calls **as if they were local function calls**
- **Abstract** the details of network communication

Remote Procedure Calls (RPC)

RPC = A mechanism for calling procedures on a remote system connected via a network

Key components:

1. Stub

- A **proxy** on the client side
- Represents the server's actual procedure
- Packs parameters and sends them to the server

2. Marshalling

- Converting data into a format suitable for network transmission
- Resolves data representation differences between different architectures
  - Big-endian vs little-endian
- Uses standards such as **XDR (External Data Representation)**

RPC Execution Flow (Figure 3.29)

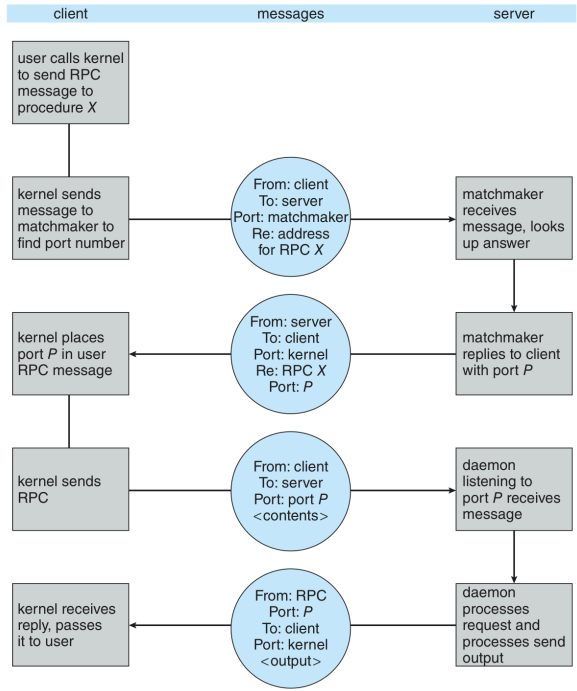


Figure 3.29 Execution of a remote procedure call (RPC).

Silberschatz, Figure 3.29 – Execution of a remote procedure call (RPC)

RPC: Execution Semantics

RPC can fail or be executed redundantly due to network errors.

"At most once"

- Attach a **timestamp** to each message
- The server maintains a history of processed timestamps
- Duplicate messages are **ignored**
- Even if the client sends multiple times, execution occurs at most once

"Exactly once"

- "At most once" + **ACK (acknowledgment)**
- The server sends an ACK to the client after completing execution
- The client **periodically retransmits** until it receives an ACK
- The most ideal but most complex to implement

RPC: Binding (Port Binding)

How does the client know the server's port number?

Method 1: Fixed port addresses

- Assign a fixed port number to the RPC at compile time
- Simple but inflexible – the server cannot change its port

Method 2: Dynamic binding (Rendezvous / Matchmaker)

- The OS provides a **matchmaker daemon** on a fixed RPC port
- Procedure:
  1. Client → matchmaker: "What is the port address for RPC X?"
  2. Matchmaker → Client: "It is port P"
  3. Client → port P: actual RPC call
- Initial overhead but provides **flexibility**

Android RPC

Android uses RPC as IPC within the same system through the **binder framework**

Service component:

- No UI, runs in the background
- A client binds to a service with `bindService()`
- Communicates via message passing or RPC

AIDL (Android Interface Definition Language):

```
/* RemoteService.aidl */
interface RemoteService {
    boolean remoteMethod(int x, double y);
}
```

- Stub code is automatically generated from the AIDL file
- The client calls it like a local method:

```
service.remoteMethod(3, 0.14);
```

- Internally, the binder framework handles marshalling and inter-process delivery

Part 7

Lab – pipe, POSIX Shared Memory

Hands-on IPC Implementation

Lab 1: Passing Data Between Parent and Child with pipe()

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define READ_END  0
#define WRITE_END 1

int main() {
    int fd[2];
    char write_msg[] = "Hello from parent!";
    char read_msg[100];

    pipe(fd);

    if (fork() == 0) {
        /* Child: read from the pipe */
        close(fd[WRITE_END]);
        read(fd[READ_END], read_msg, sizeof(read_msg));
        printf("Child received: %s\n", read_msg);
        close(fd[READ_END]);
    } else {
```

Lab 1: Execution Flow Analysis

```
1. pipe(fd) → fd[0]=read, fd[1]=write created

2. fork() → both parent and child have fd[0], fd[1]

3. Parent:
   - close(fd[READ_END])    /* close read end */
   - write(fd[WRITE_END], "Hello from parent!", 19)
   - close(fd[WRITE_END])   /* close write end */
   - wait(NULL)             /* wait for child to terminate */

4. Child:
   - close(fd[WRITE_END])   /* close write end */
   - read(fd[READ_END], ... )/* receive data sent by parent */
   - printf → "Child received: Hello from parent!"
   - close(fd[READ_END])
```

Key point: Unused ends **must** be closed for proper EOF detection

Lab 2: Producer-Consumer (Using pipe)

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

#define READ_END  0
#define WRITE_END 1

int main() {
    int fd[2];
    pipe(fd);

    if (fork() == 0) {
        /* Producer (child) */
        close(fd[READ_END]);
        for (int i = 0; i < 5; i++) {
            int item = i * 10;
            write(fd[WRITE_END], &item, sizeof(int));
            printf("Produced: %d\n", item);
        }
        close(fd[WRITE_END]);
    } else {
        /* Consumer (parent) */
        close(fd[WRITE_END]);
        int item;
```

Lab 2: Execution Result

Expected output:

```
Produced: 0
Produced: 10
Produced: 20
Produced: 30
Produced: 40
Consumed: 0
Consumed: 10
Consumed: 20
Consumed: 30
Consumed: 40
```

Behavior analysis:

- The producer (child) writes 0, 10, 20, 30, 40 to the pipe in order
- The consumer (parent) receives them in order using read()
- When the producer closes the write end, the consumer's `read()` returns 0 → loop terminates
- The actual output order may be interleaved depending on scheduling

Lab 3: POSIX Shared Memory – Producer

```c
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <unistd.h>

#define SHM_NAME "/os_lab_shm"
#define SHM_SIZE 4096

int main() {
    /* Create shared memory */
    int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    ftruncate(shm_fd, SHM_SIZE);

    /* Map to memory */
    void *ptr = mmap(0, SHM_SIZE,
                     PROT_READ | PROT_WRITE,
                     MAP_SHARED, shm_fd, 0);

    /* Write data */
    sprintf(ptr, "Operating Systems is fun!");
    printf("Producer: wrote to shared memory\n");
```

Lab 3: POSIX Shared Memory – Consumer

```c
#include <stdio.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <unistd.h>

#define SHM_NAME "/os_lab_shm"
#define SHM_SIZE 4096

int main() {
    /* Open shared memory (read-only) */
    int shm_fd = shm_open(SHM_NAME, O_RDONLY, 0666);

    /* Map to memory */
    void *ptr = mmap(0, SHM_SIZE,
                     PROT_READ,
                     MAP_SHARED, shm_fd, 0);

    /* Read data */
    printf("Consumer read: %s\n", (char *)ptr);

    /* Cleanup */
    munmap(ptr, SHM_SIZE);
    shm_unlink(SHM_NAME);
```

Lab 3: Key API Summary

Function	Role
`shm_open(name, flags, mode)`	Create/open a shared memory object
`ftruncate(fd, size)`	Set the shared memory size
`mmap(addr, length, prot, flags, fd, offset)`	Map to the address space
`munmap(addr, length)`	Unmap
`shm_unlink(name)`	Delete the shared memory object

Key flags:

- `O_CREAT` – create if it does not exist

Lab Key Takeaways

pipe():

- Creates a unidirectional communication channel between parent and child
- Read from fd[0], write to fd[1]
- Unused ends must be closed with close()
- The child inherits the pipe through fork()

POSIX Shared Memory:

- Follow the order: `shm_open()` → `ftruncate()` → `mmap()`
- Clean up with `munmap()` + `shm_unlink()` after use
- Other processes access it using the same name

Common considerations:

- In real systems, **synchronization** is critically important
- Techniques for preventing race conditions → covered in detail in Ch 6, 7

Part 8

Summary and Next Week Preview

Summary

This week's topics:

- **Two IPC models**: Shared Memory vs Message Passing
- **Shared-Memory Systems**: Producer-Consumer, Bounded/Unbounded Buffer
  - Fast but the programmer must handle synchronization directly
- **Message-Passing Systems**: Naming (Direct/Indirect), Synchronization (Blocking/Non-blocking), Buffering (Zero/Bounded/Unbounded)
  - OS manages communication, rendezvous pattern
- **IPC implementation examples**: POSIX Shared Memory, Mach Message Passing, Windows ALPC
- **Pipes**: Ordinary Pipe (unidirectional, parent-child) vs Named Pipe (FIFO, unrelated processes)
- **Client-Server Communication**:
  - Sockets — IP + Port, TCP/UDP, Loopback (127.0.0.1)
  - RPC — Stub, Marshalling/Unmarshalling, XDR, Execution Semantics
- **Lab**: IPC programming with pipe() and POSIX shared memory

Next Week Preview

Week 4 – Threads and Concurrency (Ch 4)

- Concept and necessity of threads
- Multithread models (Many-to-One, One-to-One, Many-to-Many)
- Thread libraries (Pthreads, Windows, Java)
- Implicit Threading
- Lab: Multithreaded programming with Pthreads

Textbook: Ch 4